

MICROSOFT AZURE PURVIEW DATA GOVERNANCE

Complete Requirements, Technologies and Implementation Process

1. Project Information

Project Type	Data Governance, Data Cataloguing and Sensitive Data Discovery
Project Code	T061
Team Members	2400080251 – Narsingoju Vaishnavi 2400080253 – Sai Krishna Sri Parvathaneni 2400080254 – Bollimuntha Tej Balaji 2400080005 – Potluri Sri Hasitha
Team Size	4 students
Primary Platform	Microsoft Azure and Microsoft Purview

2. Problem Statement

Data is distributed across multiple storage and database sources. Without a centralized catalogue, users cannot easily discover assets, understand schemas, identify sensitive information or validate data ownership. The proposed system uses Microsoft Purview to register sources, scan metadata, catalogue assets, classify sensitive data and document governance findings.

3. Objectives

- Catalogue data assets from five supported data sources.
- Register sources and configure secure scanning connections.
- Extract metadata and schema information through scanning.
- Identify sensitive data locations using classifications.
- Support search and discovery through the Unified Catalog.
- Validate classifications with data owners.
- Document scan results, limitations and governance findings.

4. Scope

- In scope: source registration, credentials, scanning, metadata extraction, classification, catalogue discovery and validation.
- Out of scope: modifying production data, collecting real personal information, enforcing access policies without authorization, and exposing secrets in GitHub.
- Use synthetic or publicly permitted datasets for demonstrations.

5. Required Technologies

- Microsoft Azure Subscription – resource creation and access management.
- Microsoft Purview – Data Map, scanning, metadata ingestion, classification and catalogue.
- Azure Blob Storage – initial CSV demonstration source.
- Azure SQL Database – structured relational source.
- Azure Data Lake Storage Gen2 – data lake source.
- Azure Synapse Analytics – analytical source.
- SharePoint Online – document source, only if access is available.
- Microsoft Entra ID / Managed Identity – authentication and role-based access.
- GitHub – documentation, architecture, screenshots and validation reports.
- Microsoft Excel or CSV – scan inventory and classification validation register.

6. Functional Requirements

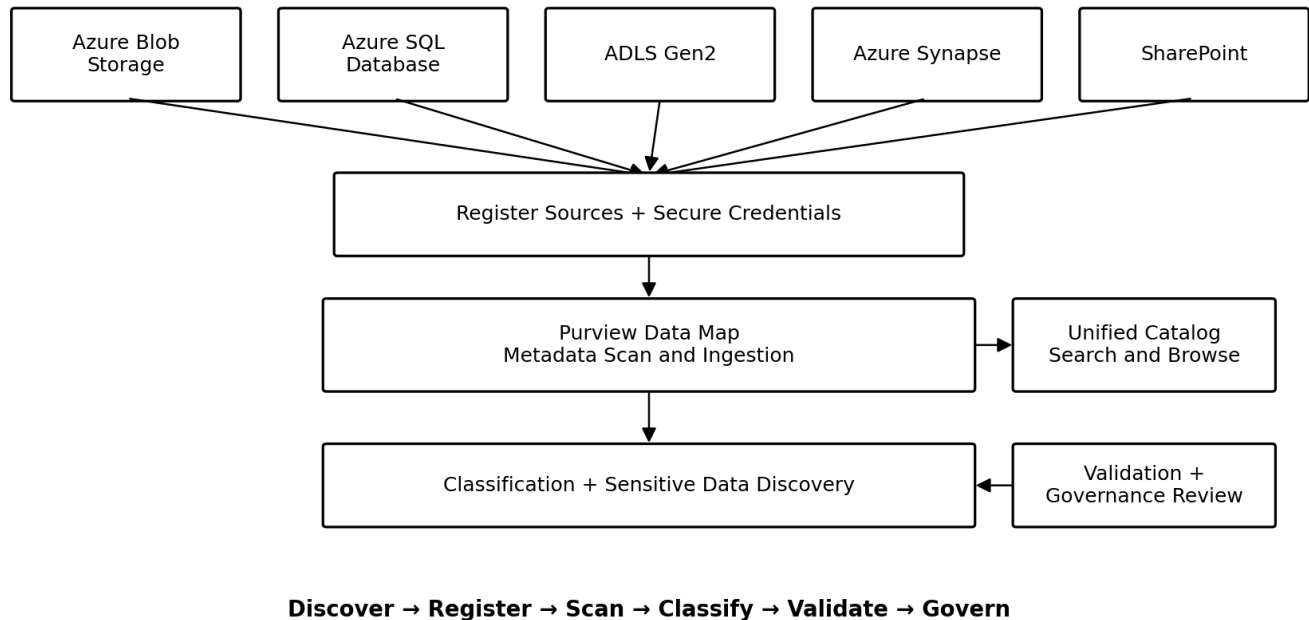
- Register supported data sources.
- Configure valid credentials and integration runtime settings.
- Scan selected folders, tables or databases.
- Ingest technical metadata and schema details.
- Apply selected scan rule sets and classifications.
- Display discovered assets in the catalogue.
- Record scan status, errors and connection failures.
- Support manual classification validation.
- Maintain evidence for each completed activity.

7. Non-Functional Requirements

- Security: never upload passwords, keys, tokens or real personal data to GitHub.
- Reliability: record failed scans and retry after permissions or network issues are corrected.
- Scalability: allow additional sources and assets to be registered later.
- Maintainability: use clear names for sources, scans, collections and files.
- Auditability: preserve screenshots, dates, scan names and validation decisions.
- Performance: scan only required folders, tables and databases during demonstration.
- Data quality: treat automatic classifications as candidates until reviewed.

8. System Architecture

AZURE PURVIEW DATA GOVERNANCE ARCHITECTURE



Data sources are registered in Purview; credentials permit controlled access; scans extract metadata; the Data Map stores discovered assets; classification identifies possible sensitive information; the Unified Catalog supports search; authorized reviewers validate results and document governance actions.

9. Step-by-Step Implementation Process

Step 1 – Prepare accounts and permissions

- Confirm an active Azure subscription.
- Confirm access to create or use a Microsoft Purview account.
- Request Purview Data Source Administrator and Data Reader permissions.
- Confirm Reader access to each Azure source that will be scanned.

Step 2 – Create or open Microsoft Purview

- Open the Azure Portal.
- Create or open a Microsoft Purview account in an available region.
- Open the Microsoft Purview governance portal.
- Create a collection such as T061-Data-Sources if collections are available.

Step 3 – Prepare synthetic data

- Create customers.csv with synthetic names, emails, phone numbers and account numbers.
- Create additional files such as employees.csv, transactions.csv and documents.csv if needed.
- Do not use real customer records.
- Upload the files to an Azure Blob Storage container such as purview-data.

Step 4 – Register Azure Blob Storage

- Open Data Map → Data Sources.
- Select Register and choose Azure Blob Storage.
- Enter a friendly source name, subscription, storage account and collection.
- Save the registration.

Step 5 – Configure and run a scan

- Open the registered source and select New Scan.
- Enter a meaningful scan name such as T061-Blob-Scan.
- Select the appropriate credential and integration runtime.
- Choose the folder or container scope.
- Select a system default or custom scan rule set.
- Test the connection.
- Choose Run once for the first demonstration.
- Select Save and Run.

Step 6 – Review results

- Open scan history and record the status.
- Open discovered assets in the Data Map or catalogue.
- Review file names, columns, schemas and classifications.
- Capture screenshots without exposing credentials or private information.

Step 7 – Validate classifications

- Create a validation sheet with Asset, Column, Detected Classification, Decision, Reviewer and Comments.
- Mark each result as Confirmed, Rejected or Pending.
- Do not treat automated classification as final without review.

Step 8 – Add other sources if access is available

- Register Azure SQL Database and configure database authentication.
- Register ADLS Gen2 and configure storage permissions.
- Register Azure Synapse Analytics if the team has access.
- Register SharePoint Online only when tenant permissions are available.
- Repeat the scan, evidence and validation process for each source.

Step 9 – Document the project in GitHub

- Create folders: docs, screenshots, sample-data and reports.
- Upload README.md, architecture diagram, implementation steps and validation template.
- Use .gitignore for secrets and local files.
- Never commit connection strings, access keys, passwords or personal data.

Step 10 – Final demonstration

- Explain the problem and objectives.
- Show the architecture.
- Show the registered data source.
- Show scan configuration and successful scan history.
- Show discovered assets and classification results.
- Explain validation, challenges and future improvements.

10. Suggested GitHub Structure

T061-Azure-Purview-Data-Governance/

```
|— README.md
|— docs/
|   |— requirements.md
|   |— implementation.md
|   |— validation-report.md
|   |— architecture.png
|— sample-data/
|   |— README.md
|— screenshots/
|   |— README.md
|— .gitignore
```

11. Evidence Checklist

- Azure subscription or resource screenshot.
- Microsoft Purview account screenshot.
- Collection screenshot, if used.
- Registered data source screenshot.
- Scan configuration screenshot.
- Test connection result.
- Successful scan history.
- Catalogue asset and schema view.
- Classification results.
- Validation sheet.
- GitHub repository and README.

12. Bottlenecks and Solutions

- Missing credentials: request authorized access or demonstrate with one accessible source.
- Permission error: verify Azure IAM and Purview collection roles.
- Network error: verify firewall, public access requirements and integration runtime.
- False classification: mark the result for business validation.
- Schema change: update scan scope or scan rule set and run a full scan when required.
- Large scan duration: use a limited folder, table or database scope for the demonstration.

13. Expected Outcome

A working demonstration of source registration, metadata scanning, catalogue discovery, sensitive-data classification and human validation using Microsoft Purview. The project will provide architecture, implementation evidence, GitHub documentation and a clear record of limitations.

14. Conclusion

Microsoft Purview supports centralized visibility into data assets across different sources. The proposed workflow—Discover, Register, Scan, Classify, Validate and Govern—provides a repeatable foundation for data governance while maintaining secure handling of credentials and synthetic demonstration data.